

1. Define Big Data. Explain at least 3 applications of Big Data.

Definition :

Big Data refers to extremely large, complex and fast growing datasets that are difficult to capture, store, manage, process and analyze using traditional data processing tools.

It requires new technologies, architectures and analytical techniques to extract value and hidden knowledge from it.

Applications of Big Data :

- ① Healthcare : Big Data helps in improving healthcare services and patient outcomes.
 - Predictive analytics is used to predict diseases and patient risks.
 - Helps in personalized treatment and medicine.
 - Analysis of large medical records, images and sensor data for better diagnosis.
- ② Banking & Finance : Big Data is widely used in the banking sector to enhance services and reduce risks.
 - Real-time fraud detection by analyzing transaction patterns.
 - Customer segmentation and personalized offers.
 - Credit scoring and risk analysis using vast customer data.
- ③ Retail & E-commerce : Big Data helps businesses understand customer behavior and improve sales.
 - Recommendation systems for products based on customer preferences.
 - Inventory management and demand forecasting.
 - Analyzing clickstream and purchase data to improve marketing strategies.

In short, Big Data helps organizations to make smarter decisions, improve efficiency, reduce costs and provide better services.

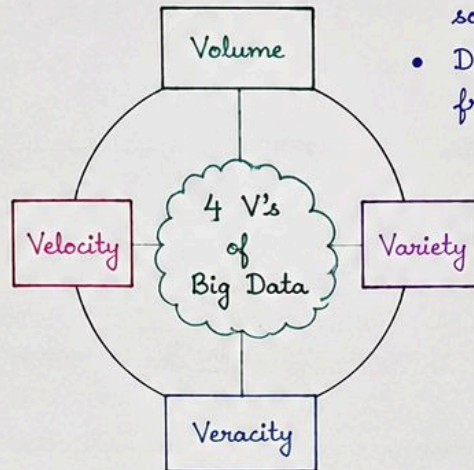
2. What are the four V's of Big Data? Explain types of Digital Data.

Four V's of Big Data :

Big Data is characterized by four key properties known as the 4 V's.

1. Velocity :

- Refers to the speed at which data is generated, collected and processed.
- High velocity data requires real-time processing and quick decision making.



1. Volume :

- Refers to the huge amount of data generated from various sources.
- Data size is continuously growing from GBs to TBs, PBs and beyond.

3. Variety :

- Refers to the different forms and types of data.
- Includes structured, semi-structured and unstructured data such as text, images, audio, video, logs, etc.

4. Veracity :

- Refers to the trustworthiness, accuracy and quality of data.
- Data may contain noise, duplicates, inconsistency and missing values, which affect reliable analysis.

Types of Digital Data :

Digital data can be broadly classified into three types.

- ① Structured Data : Data that is well-organized and follows a fixed schema or format. It is stored in rows and columns.

Examples : Relational databases, spreadsheets, transaction records, customer details.

Eg : ID, Name, Age, Salary in a table.

- ② Semi-Structured Data : Data that does not follow a rigid table structure but contains tags or markers to separate elements.

Examples : XML, JSON, logs, emails, web pages.

Eg : JSON file of a product → {"id":101, "name": "Mobile", "price":15000}

- ③ Unstructured Data : Data with no predefined structure or format. It is difficult to store and analyze using traditional methods.

Examples : Text documents, images, audio, video, social media posts.

Eg : Facebook posts, YouTube videos, images, voice recordings.

These types of data are handled using Big Data technologies like Hadoop, Spark, NoSQL databases, etc., to store, process and analyze efficiently.

3. Explain the core components of Hadoop. Draw and elaborate HDFS Architecture.

Core Components of Hadoop :

Hadoop is an open source framework for storing and processing large volumes of data in a distributed environment. Its core components are :

① **HDFS (Hadoop Distributed File System) :**

A distributed file system that stores large files across cluster nodes and provides high throughput and fault tolerance.

② **YARN (Yet Another Resource Negotiator) :**

The resource management and job scheduling layer. It manages cluster resources and allows multiple data processing engines (MapReduce, Spark, etc.) to run on the same cluster.

③ **MapReduce :**

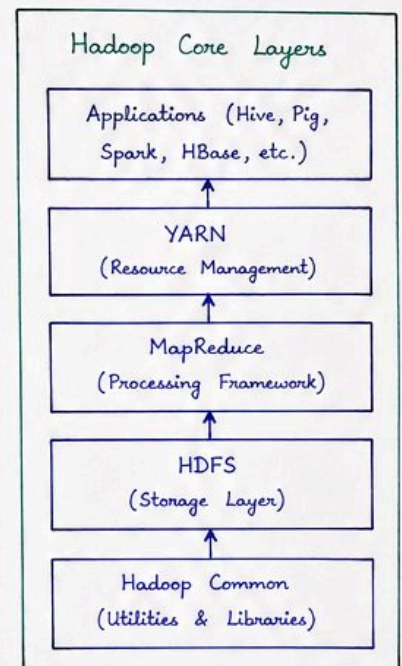
A programming model and processing engine for large scale data processing using the Map and Reduce paradigm.

④ **Hadoop Common :**

A set of common utilities and libraries that support other Hadoop modules.

⑤ **Other Ecosystem Components (optional but important) :**

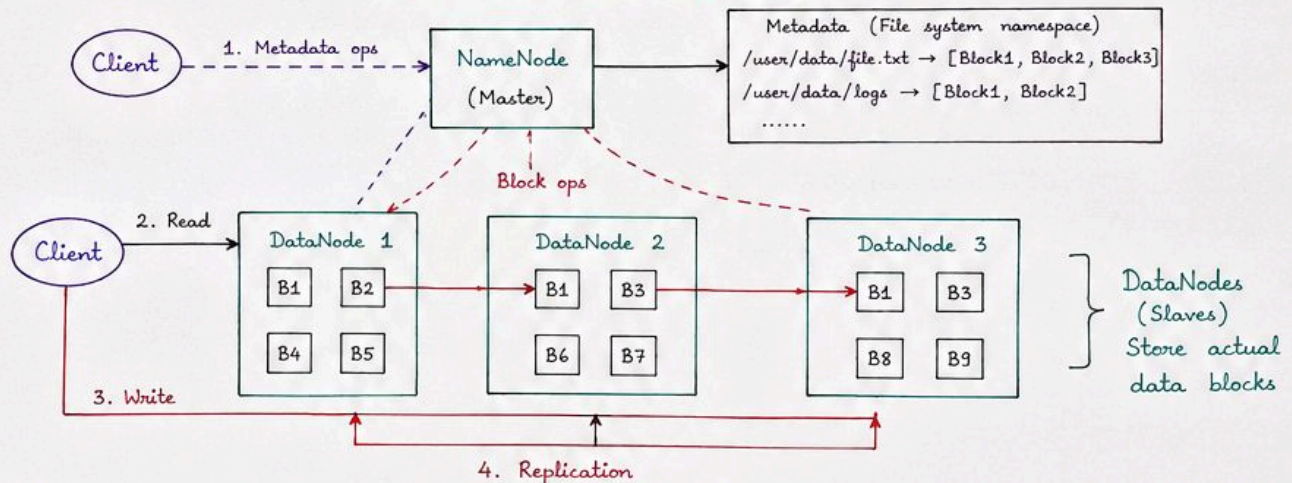
- Hive : Data warehousing and SQL like queries.
- HBase : NoSQL database on top of HDFS.
- Sqoop : Data transfer between RDBMS and Hadoop.
- Oozie : Workflow scheduling.
- ZooKeeper : Coordination service.



HDFS Architecture :

HDFS follows a master-slave architecture. It is designed to store very large files with streaming data access pattern, running on commodity hardware.

HDFS ARCHITECTURE DIAGRAM



Components :

- **NameNode (Master)** : Manages the file system namespace and metadata. It keeps track of which blocks are stored in which DataNodes.
- **DataNodes (Slaves)** : Store the actual data blocks. They serve read/write requests from clients and perform block creation, deletion and replication as per NameNode instructions.
- **Client** : Performs file operations (read/write) by communicating with NameNode for metadata and with DataNodes for actual data transfer.

Working :

- ① Client requests metadata (e.g., open file) from NameNode.
- ② NameNode returns the list of DataNodes where the required blocks are located.
- ③ Client reads/writes data directly to/from DataNodes.
- ④ Data replication is performed across multiple DataNodes (default replication factor = 3) for fault tolerance.

Key Features of HDFS :

- Fault tolerant (replication of blocks)
- High throughput access to data
- Designed for large files (GBs to TBs)
- Write once, read many
- Works on commodity hardware
- Scalable to thousands of nodes

4. Explain HDFS commands.

Hadoop provides command line interface to interact with HDFS. We use hdfs dfs command to perform various operations on files and directories in HDFS.

Syntax :

`hdfs dfs -command <arguments>`

Commonly Used HDFS Commands :

Command	Syntax	Description
• -ls	<code>hdfs dfs -ls <path></code>	Lists files and directories in the given path.
• -ls -R	<code>hdfs dfs -ls -R <path></code>	Lists files recursively in the given path.
• -mkdir	<code>hdfs dfs -mkdir <dir_path></code>	Creates a new directory in HDFS.
• -put	<code>hdfs dfs -put <local_src> <hdfs_dest></code>	Copies a file from local file system to HDFS.
• -copyFromLocal	<code>hdfs dfs -copyFromLocal <local_src> <hdfs_dest></code>	Same as -put (copy file/directory from local to HDFS).
• -get	<code>hdfs dfs -get <hdfs_src> <local_dest></code>	Copies a file from HDFS to local file system.
• -copyToLocal	<code>hdfs dfs -copyToLocal <hdfs_src> <local_dest></code>	Same as -get (copy file from HDFS to local).
• -cat	<code>hdfs dfs -cat <file></code>	Displays the contents of a file.
• -tail	<code>hdfs dfs -tail <file></code>	Displays the last 1KB of the file.
• -head	<code>hdfs dfs -head <file></code>	Displays the first 1KB of the file.
• -rm	<code>hdfs dfs -rm <file></code>	Deletes the specified file.
• -rm -r	<code>hdfs dfs -rm -r <dir_path></code>	Deletes the directory and all its contents recursively.
• -mv	<code>hdfs dfs -mv <src> <dest></code>	Moves/renames a file or directory.
• -cp	<code>hdfs dfs -cp <src> <dest></code>	Copies a file or directory within HDFS.
• -du	<code>hdfs dfs -du -h <path></code>	Shows disk usage (space used) of files/directories.
• -df	<code>hdfs dfs -df -h</code>	Shows HDFS filesystem space summary (total, used, available).
• -count	<code>hdfs dfs -count <path></code>	Shows number of directories, files and bytes under the path.
• -chmod	<code>hdfs dfs -chmod <mode> <path></code>	Changes permission (mode) of file/directory.
• -chown	<code>hdfs dfs -chown <owner:group> <path></code>	Changes owner and group of file/directory.
• -help	<code>hdfs dfs -help [command]</code>	Displays help information about HDFS commands.

Note : <path> can be any HDFS path like /user/hadoop/input or /data/file.txt
Most commands are similar to UNIX commands.

1. Discuss the anatomy of Map Reduce programming model with a neat diagram.

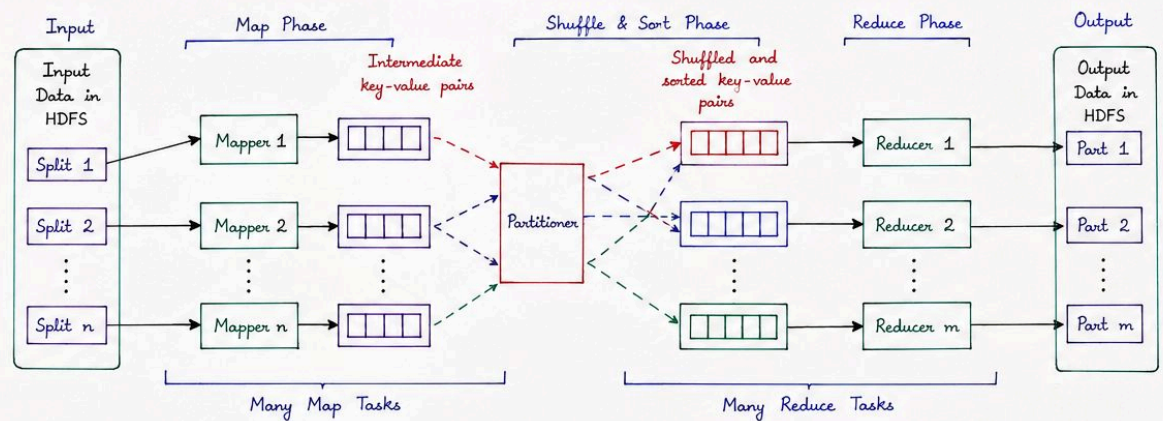
MapReduce is a programming model and processing framework for processing large scale data using parallel, distributed algorithm on a cluster.

A MapReduce job works in two main phases: Map phase and Reduce phase.

Components of MapReduce

- ① **Input Format**: Responsible for reading input data from HDFS and generating input splits.
- ② **Mapper**: Reads input splits and converts them into intermediate key-value pairs.
- ③ **Combiner (Optional)**: Performs local aggregation on mapper output to reduce data transfer.
- ④ **Partitioner**: Partitions the intermediate key-value pairs so that same key goes to the same reducer.
- ⑤ **Reducer**: Merges all values for each key and produces the final output.
- ⑥ **Output Format**: Writes the reduced output back to HDFS.

MapReduce Programming Model (Anatomy)



Working Steps:

- ① Input data stored in HDFS is divided into fixed size blocks which are further divided into Input Splits.
- ② InputFormat creates input splits and assigns them to map tasks.
- ③ Mapper reads the input split, processes the data and produces intermediate key-value pairs.
- ④ (Optional) Combiner performs local aggregation to minimize the data sent over the network.
- ⑤ Partitioner ensures that all identical keys are sent to the same reducer.
- ⑥ The Shuffle & Sort phase transfers data to reducers and sorts by key.
- ⑦ Reducer processes all values for a key and produces final output.
- ⑧ OutputFormat writes the final output to HDFS.

Key Points:

- Map tasks are embarrassingly parallel.
- Reduce tasks handle data aggregation.
- Data locality is achieved by executing map tasks close to data.
- Fault tolerance is provided by re-executing failed tasks.

2. Explain HiveQL for creating, loading data, manipulating the data in tables with an example .

Introduction to HiveQL:

HiveQL (Hive Query Language) is SQL-like language used to query, manipulate and manage data in Hive. It is built on top of Hadoop and provides data summarization and analysis through simple SQL-like queries.

A. Creating Tables (Data Definition)

- Create a database

```
CREATE DATABASE salesdb;
USE salesdb; -- to use the database
```

- Create a table

Example :

```
CREATE TABLE customers (
  cust_id      INT,
  name         STRING,
  city         STRING,
  signup_date  DATE
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;
```

→ Creates a table customers with comma(,) separated values stored as text files in HDFS.

B. Loading Data into Tables

- Load data from a local file into Hive table

```
LOAD DATA LOCAL INPATH '/user/hive/input/customers.txt'
INTO TABLE customers;
```

→ Loads data into the existing table.

- Load data with overwrite

```
LOAD DATA LOCAL INPATH '/user/hive/input/customers.txt'
OVERWRITE INTO TABLE customers;
```

- Create table and load data at once

```
CREATE TABLE orders (
  order_id     INT,
  cust_id      INT,
  amount       DOUBLE,
  order_date   STRING,
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/user/hive/warehouse/orders';
```

→ Creates table and specifies location in HDFS.

```
LOAD DATA LOCAL INPATH '/user/hive/input/orders.txt'
INTO TABLE orders;
```

→ Then loads data into orders table.

C. Manipulating Data (Data Manipulation)

1. Selecting Data

```
SELECT * FROM customers; -- all columns
SELECT name, city FROM customers; -- specific columns
SELECT * FROM customers WHERE city = 'Hyderabad';
SELECT * FROM customers WHERE signup_date >= '2024-01-01';
```

2. Filtering with Conditions

```
SELECT * FROM orders WHERE amount > 1000;
SELECT cust_id, SUM(amount) AS total_spent
FROM orders
GROUP BY cust_id; -- aggregation
```

3. Sorting Results

```
SELECT * FROM customers ORDER BY name ASC;
SELECT * FROM orders ORDER BY amount DESC;
```

4. Using Built-in Functions

```
SELECT COUNT(*) FROM customers;
SELECT MAX(amount) FROM orders;
SELECT UPPER(name) FROM customers;
```

5. Inserting Data (from another table)

```
INSERT INTO TABLE customers
SELECT * FROM new_customers;
```

Example Dataset

customers.txt		
cust_id	name, city	signup_date
1, Ravi	Hyderabad,	2024-01-10
2, Sneha	Bangalore,	2024-02-15
3, Amit	Chennai	2024-03-05
4, Pooja	Hyderabad	2024-03-20

orders.txt			
order_id	cust_id	amount	order_date
101, 1	1200.50	2024-04-01	
102, 2	850.00	2024-04-02	
103, 1	300.00	2024-04-03	
104, 3	2100.00	2024-04-04	
105, 4	500.00	2024-04-05	

Sample Query & Result

```
SELECT cust_id, SUM(amount) AS total_spent
FROM orders
GROUP BY cust_id;
```

cust_id	total_spent
1	1500.50
2	850.00
3	2100.00
4	500.00

Thus, HiveQL helps in creating tables, loading data and performing various manipulations to analyze data efficiently on big datasets.

3. Example of how Bucketing improves query performance in Hive.

What is Bucketing?

Bucketing is a technique where data in a table is divided into a fixed number of buckets based on the hash value of a specified column (bucketing column). It helps in reducing the amount of data that needs to be scanned, especially in joins and aggregations.

Example Scenario :

Suppose we have two tables in Hive :

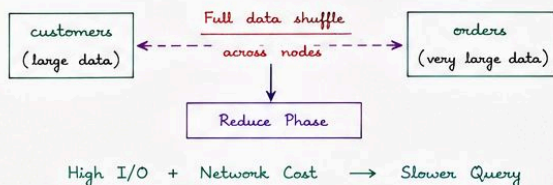
- customers (customer_id, name, city)
- orders (order_id, customer_id, amount)

Query : Find total amount spent by each customer.

```
SELECT c.customer_id, c.name, SUM(o.amount) AS total_spent
FROM customers c JOIN orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;
```

Case 1 : Without Bucketing

Both tables are stored as regular (non-bucketed) tables. During the join, Hive has to shuffle large amounts of data across the network.



Conclusion :

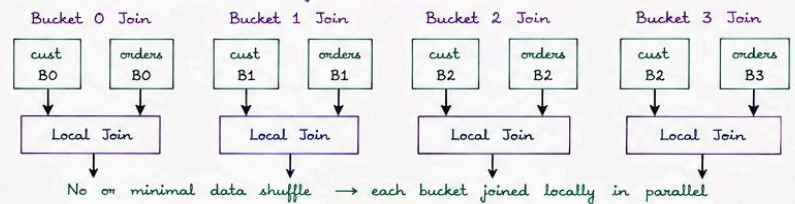
Bucketing improves query performance by reducing data shuffle, minimizing network I/O and enabling parallel local joins, especially for large tables and frequent join queries.

Case 2 : With Bucketing

Let's bucket both tables by customer_id into 4 buckets.

customers (bucketed by customer_id)				orders (bucketed by customer_id)			
Bucket 0	Bucket 1	Bucket 2	Bucket 3	Bucket 0	Bucket 1	Bucket 2	Bucket 3
c_id: 4, 8, 12	c_id: 1, 5, 9	c_id: 2, 6, 10	c_id: 3, 7, 11	orders of 4, 8, 12	orders of 1, 5, 9	orders of 2, 6, 10	orders of 3, 7, 11

Now, during the join, Hive performs bucket map join (if number of buckets is same and bucketing column is used in join).



Hive DDL for Bucketing

Create bucketed customers table

```
CREATE TABLE customers (
  customer_id INT,
  name STRING,
  city STRING
)
CLUSTERED BY (customer_id)
INTO 4 BUCKETS
STORED AS ORC;
```

Create bucketed orders table

```
CREATE TABLE orders (
  order_id INT,
  customer_id INT,
  amount DOUBLE
)
CLUSTERED BY (customer_id)
INTO 4 BUCKETS
STORED AS ORC;
```

Performance Improvement

Aspect	Without Bucketing	With Bucketing (4 buckets)
Data Shuffle	High (full data shuffle)	Low / None (bucket map join)
Network I/O	High	Very Low
Parallelism	Limited	High (per-bucket parallelism)
Query Time	Slower	Significantly Faster

4. Explain different Join strategies in Hive.

A JOIN clause is used to combine rows from two or more tables based on a related column between them.

Types of Joins in Hive

- 1 Inner Join
- 2 Left Outer Join
- 3 Right Outer Join
- 4 Full Outer Join
- 5 Left Semi Join
- 6 Map Join (Broadcast Join)
- 7 Sort Merge Bucket (SMB) Join

Example Tables

A (emp)			B (dept)	
emp_id	name	dept_id	dept_id	dept_name
1	Ravi	10	10	HR
2	Sita	20	20	Finance
3	Mohan	10	30	IT
4	Neha	30	40	Admin

5 Left Semi Join

Returns rows from left table (A) which have matching rows in right table (B). It returns only columns from left table.

```
SELECT A.*
FROM emp A LEFT SEMI JOIN dept B
ON A.dept_id = B.dept_id;
```

emp_id	name	dept_id
1	Ravi	10
2	Sita	20
3	Mohan	10
4	Neha	30

6 Map Join (Broadcast Join)

- Used when one table is small enough to fit in memory.
- The small table is broadcast to all mappers and the join is performed.
- Enabled using /*+ MAPJOIN(table_alias) */ hint.
- Faster than shuffle-based join for large data.

```
SELECT /*+ MAPJOIN(d) */ e.emp_id, e.name, d.dept_name
FROM emp e JOIN dept d
ON e.dept_id = d.dept_id;
```

7 Sort Merge Bucket (SMB) Join

- Used when both tables are bucketed and sorted on the join key.
- Avoids expensive shuffle phase.
- Most efficient when tables are large and properly bucketed.
- Requirements:
 - Both tables must be bucketed on join key with same number of buckets.
 - Bucketing and sorting order must be same.
 - Parameter "hive.optimize.bucketmapjoin=true" or "hive.enforce.bucketing=true".

Note: Hive automatically decides the best join strategy based on table size, statistics and configuration. You can influence it using hints and parameters.

1 Inner Join

Returns only matching rows from both tables.

```
SELECT A.emp_id, A.name, B.dept_name
FROM emp A JOIN dept B
ON A.dept_id = B.dept_id;
```

emp_id	name	dept_name
1	Ravi	HR
2	Sita	Finance
3	Mohan	HR
4	Neha	IT

2 Left Outer Join (Left Join)

Returns all rows from left table (A) and matching rows from right table (B). Non-matching rows in B are NULL.

```
SELECT A.emp_id, A.name, B.dept_name
FROM emp A LEFT OUTER JOIN dept B
ON A.dept_id = B.dept_id;
```

emp_id	name	dept_name
1	Ravi	HR
2	Sita	Finance
3	Mohan	HR
4	Neha	IT

3 Right Outer Join (Right Join)

Returns all rows from right table (B) and matching rows from left table (A). Non-matching rows in A are NULL.

```
SELECT A.emp_id, A.name, B.dept_name
FROM emp A RIGHT OUTER JOIN dept B
ON A.dept_id = B.dept_id;
```

emp_id	name	dept_name
1	Ravi	HR
2	Sita	Finance
3	Mohan	HR
4	Neha	IT
NULL	NULL	Admin

4 Full Outer Join (Full Join)

Returns all rows from right tables. Non-matching rows are filled with NULLs.

```
SELECT A.emp_id, A.name, B.dept_name
FROM emp A FULL OUTER JOIN dept B
ON A.dept_id = B.dept_id;
```

emp_id	name	dept_name
1	Ravi	HR
2	Sita	Finance
3	Mohan	HR
4	Neha	IT
NULL	NULL	Admin

Summary of Join Strategies

Join Type	Returns	Use When
Inner Join	Only matching rows	Want only matched data
Left Outer Join	All from left + matched from right	Keep all left data
Right Outer Join	All from right + matched from left	Keep all right data
Full Outer Join	All from both sides	Need complete data
Left Semi Join	Left rows having match in right	Existence check / filtering
Map Join	Join using in-memory broadcast	One table is small
SMB Join	Join using sorted bucketed tables	Both tables large & bucketed

1. Describe the process of importing data from a relational database into Hadoop's HDFS using Sqoop.

Sqoop : SQL to Hadoop and Hadoop to SQL.

Sqoop is a tool used to transfer data between relational databases (e.g., MySQL, Oracle, PostgreSQL) and Hadoop's HDFS.

Import : Data from RDBMS → HDFS/Hive/HBase

Process of Importing Data from RDBMS to HDFS

① Connect to Database and Fetch Metadata

Sqoop connects to the relational database using JDBC, fetches the metadata (table structure, column types, etc.) of the specified table.

② Generate Map Tasks (Splits)

Based on the table and the split column, Sqoop divides the data into multiple chunks (input splits). Each split is assigned to a Map task to enable parallel import.

③ Launch Map Tasks

Sqoop launches multiple Map tasks. Each task runs a SQL query on its assigned split to extract a portion of the data from the database.

④ Transfer Data to HDFS

Each Map task writes the fetched rows as text files (or sequence files) and stores them in the specified directory in HDFS.

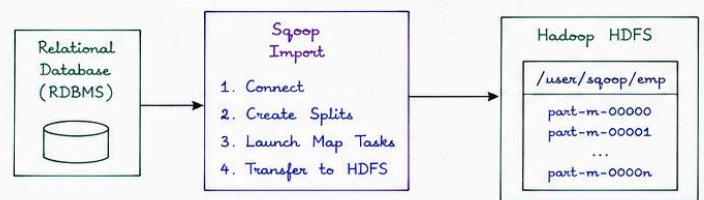
⑤ Completion

After all Map tasks finish, the data from the table is successfully imported into HDFS.

Key Points :

- Sqoop uses MapReduce to perform import in parallel.
- The number of mappers depends on the number of splits.
- Default file format in HDFS is text.

Architecture of Sqoop Import



Example : Import table 'emp' from MySQL to HDFS

```
sqoop import \  
--connect jdbc:mysql://localhost:3306/company \  
--username root --password root \  
--table emp \  
--target-dir /user/sqoop/emp \  
--split-by emp_id --num-mappers 4 \  
--fields-terminated-by ',' --lines-terminated-by '\n'
```

Explanation of important options :

- --connect : JDBC connection string to the database.
- --username, --password : Database credentials.
- --table : Table to import.
- --target-dir : HDFS directory to store imported data.
- --split-by : Column used to split data for parallel import.
- --num-mappers : Number of parallel tasks.
- --fields-terminated-by : Delimiter for fields (columns).

Result in HDFS (/user/sqoop/emp)

```
$ hdfs dfs -ls /user/sqoop/emp  
Found 4 items  
-rw-r--r-- 3 hadoop super 1024 2024-05-20 10:20 /user/sqoop/emp/part-m-00000  
-rw-r--r-- 3 hadoop super 1083 2024-05-20 10:20 /user/sqoop/emp/part-m-00001  
-rw-r--r-- 3 hadoop super 1112 2024-05-20 10:20 /user/sqoop/emp/part-m-00002  
-rw-r--r-- 3 hadoop super 998 2024-05-20 10:20 /user/sqoop/emp/part-m-00003
```

Note : After import, the data can be queried using tools like Hive or Pig, or further processed using MapReduce.

2. Overview of Exporting Data from HDFS /Hive to a Relational Database using Sqoop

Sqoop can transfer data from Hadoop (HDFS files or Hive tables) back to a relational database table. It performs the export in parallel using MapReduce jobs.

Architecture



How Export Works

- 1 **Connect**: Sqoop connects to the target relational database using JDBC and validates the connection.
- 2 **Create (Optional)**: Sqoop can create the target table in the RDBMS if it does not exist.
- 3 **Split**: Sqoop determines how to divide the data into multiple chunks (splits) so that many mappers can export data in parallel. Splits are based on a numeric column (e.g., primary key) or query-based splitting.
- 4 **Map Tasks**: Each mapper reads a portion of the data from HDFS/Hive and writes rows to the target table using JDBC batch inserts.
- 5 **Completion**: When all mappers finish, the export job completes.

Export Sources

- From HDFS: Data in a directory (text or sequence files, AVRO, etc.)
- From Hive: Data from a Hive table or the result of a Hive query.

Key Points

- Sqoop uses MapReduce to do parallel exports.
- The export column must be a numeric column (e.g., ID) to determine splits.
- Data is inserted using JDBC batch operations for better performance.
- Constraints (primary keys, unique keys) should be considered to avoid duplicates.
- Large exports benefit from tuning: number of mappers, batch size, direct mode.

Basic Export Syntax

(a) Export from HDFS to RDBMS

```
sqoop export \  
--connect <jdbc-url> \  
--username <user> --password <pwd> \  
--table <target-table> \  
--export-dir <hdfs-dir> \  
--input-fields-terminated-by '\t' \  
--num-mappers <N> \  
--batch
```

(b) Export from Hive table to RDBMS

```
sqoop export \  
--connect <jdbc-url> \  
--username <user> --password <pwd> \  
--table <target-table> \  
--hive-table <db.table> \  
--hive-export-dir <hdfs-tmp-dir> \  
--num-mappers <N> \  
--batch
```

Example

1. Export from HDFS (employee.txt) to MySQL table emp

```
sqoop export \  
--connect jdbc:mysql://localhost:3306/company \  
--username root --password root \  
--table emp \  
--export-dir /user/hive/warehouse/emp/ \  
--input-fields-terminated-by ',' \  
--num-mappers 4 --batch
```

MySQL Table : emp

id	name	dept	salary
1	Ravi	IT	50000
2	Sita	HR	45000
3	Mohan	IT	60000
...			

2. Export from Hive table to Oracle table emp

```
sqoop export \  
--connect jdbc:oracle:thin:@orcl:1521:xe \  
--username scott --password tiger \  
--table emp \  
--hive-table default.emp \  
--hive-export-dir /tmp/hive-emp-export \  
--num-mappers 8 --batch
```

Oracle Table : emp

id	name	dept	salary
1	Ravi	IT	50000
2	Sita	HR	45000
3	Mohan	IT	60000
...			

Important Options

Option	Purpose
--export-dir	HDFS directory containing data to export (for HDFS exports).
--hive-table	Hive table to export.
--hive-export-dir	Temporary HDFS directory where Hive data is dumped before export.
--num-mappers	Number of parallel tasks (mappers).
--input-fields-terminated-by	Field delimiter for text data.
--batch	Use JDBC batch updates for better performance.
--columns/--export-columns	Specify columns to export.
--where	Export only rows matching the condition (applies to HDFS exports).

Note: Sqoop export is efficient for bulk data movement from Hadoop/Hive to RDBMS. For very large exports, tune the number of mappers, batch size, and ensure proper indexing on target tables.

3. HBase Architecture in Detail

Overview

HBase is a distributed, column-oriented, NoSQL database built on top of HDFS. It provides random, real-time read/write access to big data. HBase follows a master-slave architecture.

Core Components

1. Client

Applications use HBase Client APIs (Java, REST, Thrift) to read/write data. The client locates the region server (holding the target region) via ZooKeeper and communicates with it.

2. ZooKeeper

HBase uses Apache ZooKeeper for coordination.

- Maintains the location of HBase meta region
- Tracks live region servers
- Helps in failover of master and region servers
- Stores configuration and state information

3. HMaster (Master Server)

- Manages the table schema and namespace.
- Assigns regions to region servers.
- Handles load balancing.
- Monitors region servers.
- Performs split of regions.
- Coordinates schema changes.

4. RegionServer

It is the worker node that serves read/write requests.

- Hosts multiple regions.
- Responsible for flush (memstore to StoreFiles).
- Handles compactions.
- Communicates with HDFS for data storage.
- Sends heartbeats to HMaster.

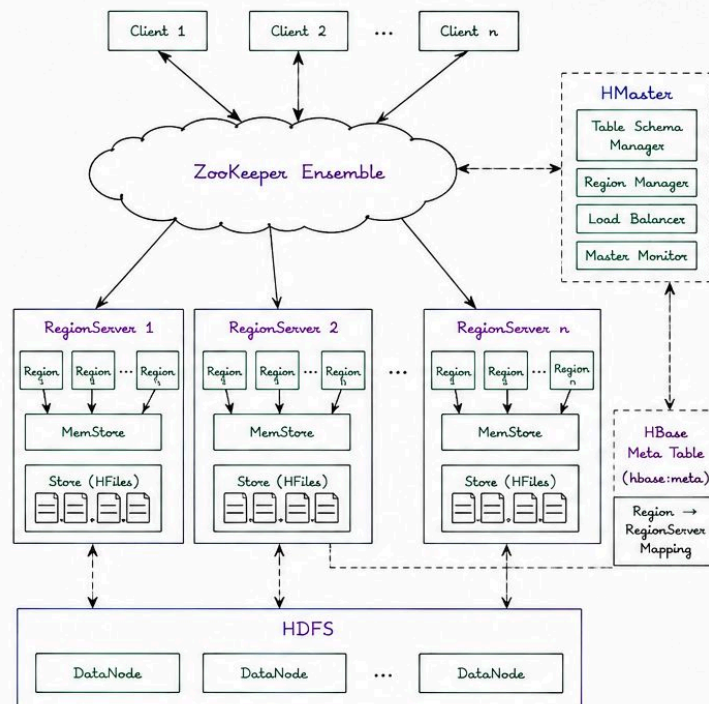
Data Flow (Write Path)

Client → ZooKeeper (find RS) → RegionServer → WAL (append) → MemStore → Flush → HFiles in HDFS

Data Flow (Read Path)

Client → ZooKeeper (find RS) → RegionServer → MemStore → (if not found) → HFiles in HDFS.

HBase Architecture



5. HRegion (Region)

- A table is divided into multiple regions (row key range based).
- Each region is a horizontal partition of the table.
- Regions are the unit of distribution and load balancing.

6. Store

- Each column family in a region is a Store.
- Stores are sorted by row key.
- Data is stored in HFiles on HDFS.

7. MemStore

- In-memory buffer for recent edits (per Store).
- When it reaches a threshold, it is flushed to an HFile in HDFS.

8. HDFS (Storage Layer)

- All actual data (HFiles), write-ahead logs (WAL), and metadata are stored in HDFS.
- Provides fault tolerance and high throughput.

9. Write-Ahead Log (WAL)

- Every write (Put/Delete) is first appended to the WAL.
- Ensures durability. If a RegionServer fails, edits are recovered from WAL.
- WAL files are stored in HDFS.

10. hbase:meta Table

- A special system table that holds the mapping of Region → RegionServer.
- Located in HDFS.
- The location of this table is stored in ZooKeeper.

Key Points

- HBase is strongly consistent at row level.
- Automatically scales by adding RegionServers.
- Provides real-time random access to big data.
- Built-in compression, versioning of data.
- Suited for sparse data (many columns).

Use Cases

- Time series data
- Click stream data
- IoT and sensor data
- User profiles, messaging, counters
- Log analytics, entity data storage

4. CRUD Operations on HBase Table

HBase Table : student

- Row Key : rowkey (string)
- Column Families : info, marks

HBase Shell
start hbase shell
hbase(main):001:0>

Table Structure

Row Key (rowkey)	info:name	info:age	info:city	marks:maths	marks:science
1001	Ravi	21	Bangalore	90	85
1002	Sita	22	Pune	88	92
1003	Mohan	20	Chennai	75	80

1. CREATE - Insert data into table

```
# Put (Insert)
hbase(main):002:0> put 'student', '1001', 'info:name', 'Ravi'
hbase(main):003:0> put 'student', '1001', 'info:age', '21'
hbase(main):004:0> put 'student', '1001', 'info:city', 'Bangalore'
hbase(main):005:0> put 'student', '1001', 'marks:maths', '90'
hbase(main):006:0> put 'student', '1001', 'marks:science', '85'

# Insert multiple columns in a single put
hbase(main):007:0> put 'student', '1002', {
  'info:name' => 'Sita',
  'info:age' => '22',
  'info:city' => 'Pune',
  'marks:maths' => '88',
  'marks:science' => '92'
}
```

2. READ - Retrieve data from table

```
# Get (read a single row)
hbase(main):008:0> get 'student', '1001'
COLUMN      CELL
info:age     timestamp=17161111111, value=21
info:city    timestamp=17161111111, value=Bangalore
info:name     timestamp=17161111111, value=Ravi
marks:maths   timestamp=17161111111, value=90
marks:science timestamp=17161111111, value=85
1 row(s) in 0.0180 seconds

# Scan (read all rows)
hbase(main):009:0> scan 'student'
# (returns all rows as in the Table Structure above)
```

3. UPDATE - Modify existing data (Put overwrites)

```
# Update a column value
hbase(main):010:0> put 'student', '1001', 'info:age', '22'
# (Age of 1001 is updated from 21 to 22)

# Update multiple columns
hbase(main):011:0> put 'student', '1002', {
  'info:city' => 'Mumbai',
  'marks:maths' => '95'
}

# (City and Maths marks of row 1002 updated)
```

4. DELETE - Remove data

```
# Delete a specific column
hbase(main):012:0> delete 'student', '1001', 'marks:science'
# (Only marks:science column of row 1001 deleted)

# Delete all versions of a column
hbase(main):013:0> deleteall 'student', '1001', 'info:city'
# (All versions of column info:city deleted)

# Delete an entire row
hbase(main):014:0> deleteall 'student', '1003'
# (Row with rowkey 1003 deleted)
```

After Above Operations

Row Key	info:name	info:age	info:city	marks:maths	marks:science
1001	Ravi	22	-	90	-
1002	Sita	22	Mumbai	95	92
1003	(Deleted)				

Useful Commands

- list 'student' - List all rows
- count 'student' - Count rows
- disable 'student' - Disable table
- drop 'student' - Drop table

Summary of CRUD

CREATE : put 'table', 'rowkey', 'cf:column', 'value'

READ : get 'table', 'rowkey' | scan 'table'

UPDATE : put on existing row/column (overwrites)

DELETE : delete 'table', 'rowkey', 'cf:column'

deleteall to remove all versions or entire row

- Notes :**
- HBase is schema-less. Column families must exist, but columns (qualifiers) are created dynamically.
 - All operations are done on row level. Data is versioned internally using timestamps.

1. Architecture of Apache Spark

Apache Spark follows a master-slave architecture. A Spark application runs as a set of independent processes that coordinate with each other to perform parallel computation on a cluster.

1. Driver Program

- The main process that runs the user's main() function.
- Creates SparkContext / SparkSession.
- Converts user code (transformations, actions) into DAG (Directed Acyclic Graph) of stages.
- Requests resources from the Cluster Manager.
- Schedules tasks to Executors.
- Monitors task execution and handles failures.

2. Cluster Manager

- Manages resources in the cluster.
- Allocates resources (CPU, memory) to Spark applications.
- Examples: Standalone Manager, Apache YARN, Mesos, Kubernetes.

3. Executors

- Worker processes launched on cluster nodes.
- Run tasks assigned by the Driver.
- Store and cache data in memory/disk.
- Return results to the Driver.

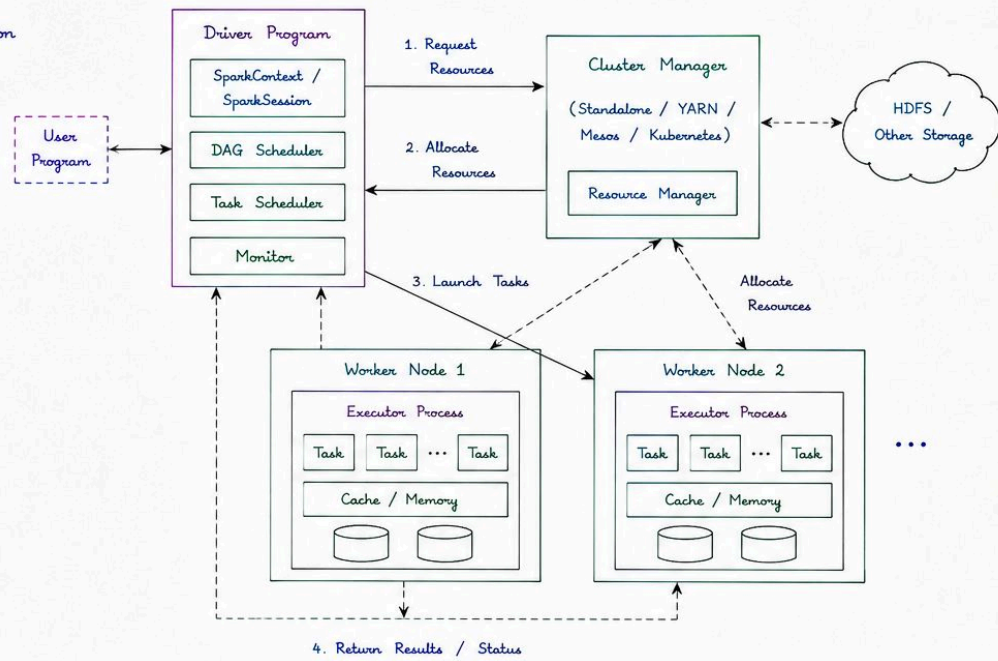
4. Worker Nodes

- Machines in the cluster that run Executor processes and store data.

How Spark Processes Data ?

- ① The user program creates RDDs/DataFrames and applies transformations.
- ② When an action is called, Driver builds a DAG of operations.
- ③ DAG is divided into stages based on shuffle dependencies.
- ④ Driver asks Cluster Manager for resources (executors).
- ⑤ Cluster Manager allocates resources on worker nodes.
- ⑥ Driver sends tasks (stages) to executors.
- ⑦ Executors read data (from memory/disk/HDFS), perform tasks in parallel.
- ⑧ Results are returned to Driver or written to storage.

Spark Architecture



Collaboration Summary

- Driver = Brain of the application (scheduling, coordination, monitoring).
- Cluster Manager = Resource allocator in the cluster.
- Executors = Do the actual computation and cache data.
- Worker Nodes = Provide compute and storage resources.
- Together they enable parallel, fault-tolerant, in-memory data processing.

Key Points

- Spark uses in-memory computing for speed.
- Data is partitioned and processed in parallel across executors.
- If an executor fails, tasks are resubmitted to other executors.
- Supports batch, streaming, SQL, ML, graph workloads.

2. What is an RDD? Illustrate how to create an RDD.

What is an RDD?

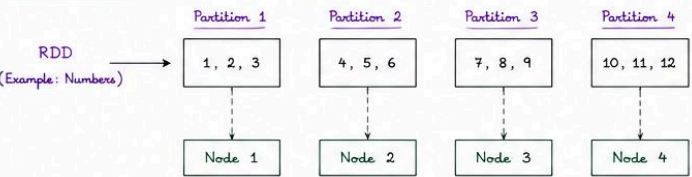
RDD (Resilient Distributed Dataset) is the fundamental data abstraction in Apache Spark.

- It is an immutable, partitioned collection of elements distributed across the cluster.
- Each RDD is divided into partitions which can be computed on different nodes in parallel.
- RDDs can be created from data in memory, files in HDFS, or other RDDs through transformations.

Key Properties of RDD

- **Immutable** - Once created, an RDD cannot be changed.
- **Distributed** - Data is split into partitions across cluster nodes.
- **Resilient** - RDDs can recover lost partitions using lineage.
- **Lazy** - Transformations are not executed until an action is called.
- **Parallel** - Operations on RDDs are executed in parallel.

RDD Structure



RDD vs DataFrame

Feature	RDD	DataFrame
Type	Unstructured (any type)	Structured (Rows & Columns)
Schema	No schema	Schema is defined
Optimization	No (low level)	Yes (Catalyst optimizer)
Ease of use	Low level API	High level API (SQL like)
Performance	Good	Better (optimized)

How to create an RDD?

RDDs in Spark can be created in two ways:

1. Parallelizing an existing collection in the driver program.
2. Reading data from external storage (like HDFS, local file, database etc.).

1. Create RDD from an existing collection (using parallelize)

```
# Scala Example
val data = List(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
val rdd = sc.parallelize(data, numSlices = 3) // 3 partitions
rdd.collect() // Output: Array(1,2,3,4,5,6,7,8,9,10)

# Python Example
data = [1,2,3,4,5,6,7,8,9,10]
rdd = sc.parallelize(data, 3) # 3 partitions
rdd.collect() # [1,2,3,4,5,6,7,8,9,10]
```

Resulting RDD
(3 partitions)

P0: [1, 2, 3, 4]

P1: [5, 6, 7]

P2: [8, 9, 10]

2. Create RDD from an external source

(a) From text file in HDFS/Local

```
# Scala
val rdd = sc.textFile("hdfs://namenode:9000/data/file.txt")

# Python
rdd = sc.textFile("hdfs://namenode:9000/data/file.txt")
```

(b) From CSV File

```
# Python
rdd = sc.textFile("data.csv") \
    .map(lambda line: line.split(","))
```

(c) From Sequence File (Hadoop)

```
# Scala
val rdd = sc.sequenceFile[Int, String]("hdfs://namenode:9000/data/file.seq")
```

Note:

- sc is the SparkContext.
- Transformations (like map, filter) on RDDs are lazy.
- Actions (like collect, count, take) trigger execution.

3. Transformations and Actions applied on an RDD with examples

RDD Operations are of two types :

- 1. Transformations (create new RDD)
- 2. Actions (return result to driver / write output)

1. TRANSFORMATIONS (Lazy Evaluation)

- They create a new RDD from an existing RDD.
- They are lazy i.e., not executed immediately.
- Execution happens only when an action is called.

Transformation	Description	Example (Scala)	Example
• map(f)	Applies function f to each element	val rdd2 = rdd.map(x => x*2)	Input: (1,2,3) Output: (2,4,6)
• filter(f)	Selects elements that satisfy condition f	val rdd2 = rdd.filter(x => x%2==0)	Input: (1,2,3,4) Output: (2,4)
• flatMap(f)	Similar to map, but flattens the results	val rdd2 = rdd.flatMap(x => x.toString.split(" "))	Input: ("a b", "c d") Output: (a,b,c,d)
• mapPartitions(f)	Applies f to each partition (iterator)	val rdd2 = rdd.mapPartitions(it => it.map(_*10))	Each partition's elements * 10
• union(rdd2)	Returns RDD with elements from both	val rdd2 = rdd1.union(rdd2)	(1,2) U (3,4) Output: (1,2,3,4)
• intersection(rdd2)	Common elements in both RDDs	rdd1.intersection(rdd2)	(1,2,3) ∩ (2,3,4) Output: (2,3)
• distinct()	Removes duplicate elements	rdd.distinct()	(1,2,2,3,3) Output: (1,2,3)
• groupByKey()	Groups values by key (for (K,V) RDD)	rdd.groupByKey()	((a,1),(a,2),(b,1)) Output: (a,[1,2]),(b,[1])
• reduceByKey(f)	Reduces values by key using function f	rdd.reduceByKey(_+_)	((a,1),(a,2),(b,1)) Output: (a,3),(b,1)
• sortBy(f)	Sorts elements based on function f	rdd.sortBy(x => x)	(3,1,2) Output: (1,2,3)
• repartition(n)	Changes number of partitions (wide)	rdd.repartition(4)	RDD with 4 partitions
• coalesce(n)	Reduces partitions (narrow)	rdd.coalesce(2)	RDD with 2 partitions

TRANSFORMATION vs ACTIONS

Transformations	Actions
• Lazy (not executed immediately) • Return a new RDD • Can be chained • Example: map, filter, union	• Trigger execution • Return final result / write output • Bring data to driver or storage • Example: collect, count, saveAsTextFile

2. ACTIONS (Trigger Execution)

- They trigger the computation and return result / output to the driver program or external storage.
- After an action is executed, transformations are recomputed if needed.

Action	Description	Example (Scala)	Example
• collect()	Returns all elements to driver as Array	val arr = rdd.collect()	(1,2,3) Array(1,2,3)
• count()	Returns number of elements	val c = rdd.count()	(1,2,3,4) Output: 4
• first()	Returns first element of RDD	val f = rdd.first()	(1,2,3) Output: 1
• take(n)	Returns first n elements	val t = rdd.take(2)	(1,2,3,4) Output: (1,2)
• takeSample(withReplacement, num, seed)	Returns random sample	rdd.takeSample(false, 2, 42)	Random 2 elements
• reduce(f)	Applies function f to all elements	val s = rdd.reduce(_+_)	(1,2,3,4) Output: 10
• fold(zeroValue)(f)	Like reduce, with initial value	rdd.fold(0)(_+_)	(1,2,3) Output: 6
• aggregate(zero)(seqOp, combOp)	Aggregate with two functions	rdd.aggregate(0)(_+_, _+_)	(1,2,3) Output: 6
• foreach(f)	Applies function f to each element	rdd.foreach(println)	Prints each element
• saveAsTextFile(path)	Saves RDD as text file in HDFS/Local	rdd.saveAsTextFile("/out/rdd")	Output files created
• countByKey()	Count of each key (for (K,V) RDD)	rdd.countByKey()	((a,1),(a,2),(b,1)) Output: (a,2),(b,1)

Example Program (Scala)

```
val sc = new SparkContext("local[*]", "RDD Example")
val rdd = sc.parallelize(1 to 5) // RDD:(1,2,3,4,5)
val rdd2 = rdd.map(x => x*2).filter(_%2==0) // Transformation (Lazy)
val result = rdd2.collect() // Action (Triggers execution)
println(result.mkString(", ")) // Output: 4,8
```

Key Points

- Transformations build the lineage (DAG).
- Action triggers the job.
- Spark recomputes lost partitions using lineage.
- Prefer transformations over actions for performance.

4. RDD Persistence (Caching) in Apache Spark

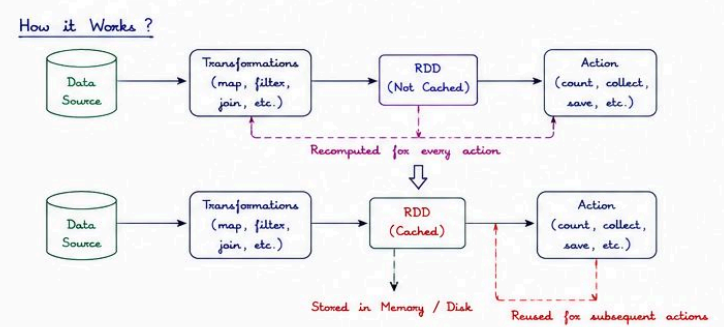
What is RDD Persistence ?

By default, RDDs in Spark are immutable and their transformations are computed every time an action is invoked.

Persistence (or caching) allows us to store an RDD in memory and/or disk so that it can be reused in multiple actions without recomputing the entire lineage (DAG) every time.

It is especially useful when the same RDD is used in multiple actions or in iterative algorithms.

- Why is it Important ?
- Avoids recomputation of expensive transformations.
 - Saves time for iterative algorithms (e.g., ML, graph processing).
 - Reduces I/O and network overhead.
 - Improves overall performance of Spark applications.



- How to Persist (Cache) an RDD ?
- persist() or cache() method is used.
 - Cache() is just a shorthand for persist(StorageLevel.MEMORY_ONLY).

```
val rdd = sc.textFile("hdfs://data/logs.txt") // Transformation
val cleaned = rdd.filter(_.contains("ERROR")) // Mark for caching
cleaned.cache() // persist() method (default) // Computed & cached
cleaned.count() // first action - triggers computation // Reused from cache
cleaned.saveAsTextFile("/output/errors") // second action - uses cached RDD
```

Storage Levels for Persistence

Spark provides different storage levels to control how RDDs are cached.

Storage Level	Description
MEMORY_ONLY	Store serialized RDD in memory. Fastest, but may not fit in memory.
MEMORY_AND_DISK	Store in memory; if not enough memory, overflow to disk.
MEMORY_ONLY_SER	Store serialized RDD in memory (more space-efficient).
MEMORY_AND_DISK_SER	Store serialized in memory, overflow to disk.
DISK_ONLY	Store only on disk.
DISK_ONLY_SER	Store serialized RDD on disk.
OFF_HEAP	Store in Tachyon/OFF-heap memory (if enabled).
NONE	Do not persist.

Example with Different Storage Levels

```
import org.apache.spark.storage.StorageLevel
val data = sc.textFile("hdfs://data/bigfile.txt")
data.persist(StorageLevel.MEMORY_AND_DISK_SER)
data.count() // triggers computation & stores
```

- Key Points
- Persistence is lazy. Data is cached only when an action is executed.
 - Cached RDDs are kept until unpersisted or evicted from memory.
 - Use unpersist() to remove the RDD from cache when it is no longer needed.
 - Caching uses cluster memory; cache only what fits and is reused.

Unpersisting an RDD

```
cleaned.unpersist() // removes from memory/disk
```

- When to Use Caching ?
- When an RDD is used in multiple actions.
 - In iterative algorithms (e.g., PageRank, KMeans).
 - When lineage is very long and recomputation is expensive.

1. Spark SQL : Concept and Role in querying structured data

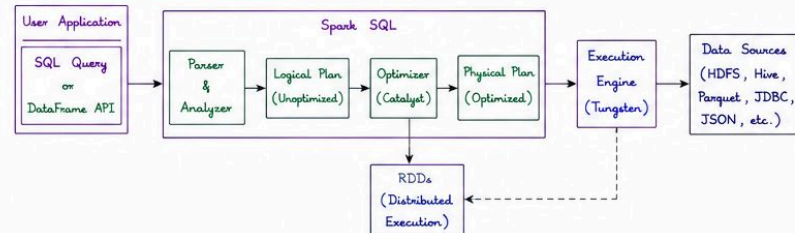
* Concept of Spark SQL

Spark SQL is a module of Apache Spark for working with structured data. It provides a programming abstraction called DataFrame and allows us to query data using SQL-like syntax. It combines the power of SQL (ease of use) with the scalability and speed of Spark (distributed processing).

* Key Points

- Spark SQL is built on top of Spark Core.
- It provides DataFrame and Dataset APIs.
- Supports SQL queries, HiveQL, and integration with Hive.
- Automatically performs query optimization.
- Works with various data sources (e.g., JSON, Parquet, CSV, JDBC, Hive tables).

* Architecture / Internal Working



- » Catalyst Optimizer : Optimizes the logical plan using rule-based and cost-based optimization.
- » Tungsten Engine ; Executes the physical plan efficiently using whole-stage code generation, memory management, and CPU-level optimizations.

* Summary

Spark SQL brings the power of SQL to big data. It enables querying and manipulating structured data at scale using familiar SQL syntax while leveraging Spark's speed and distributed capabilities.

* Role of Spark SQL

1. Query Structured Data : Allows users to query structured data using standard SQL syntax.
2. Unified Data Access ; Works with multiple data sources using a single API.
3. Performance Optimization ; Uses Catalyst optimizer and Tungsten engine for fast execution.
4. Integration : ; Easily integrates SQL with other Spark components (Streaming, MLlib, GraphX).

* Example

1. Using SQL on DataFrame

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("SparkSQLDemo").getOrCreate()
df = spark.read.option("header", True).csv("employees.csv", inferSchema=True)
df.createOrReplaceTempView("emp")

# SQL Query
result = spark.sql("SELECT dept, AVG(salary) AS avg_sal
FROM emp GROUP BY dept")

result.show()
```

2. DataFrame API (Equivalent)

```
from pyspark.sql.functions import avg
result_df = df.groupBy("dept").agg(avg("salary").alias("avg_sal"))
result_df.show()
```

* Features / Advantages

Feature	Benefit
SQL pyspark	Easy to use for analysts & developers
Catalyst Optimizer	Automatically optimizes queries
Tungsten Execution Engine	High performance & memory efficiency
Multiple Data Sources	Flexibility to work with diverse data
Seamless Integration	Works with Spark Core, Streaming, MLlib, Hive

2. Aggregation Functions in Spark

* Definition

Aggregation functions are operations that compute a summary value from a collection of values. In Spark, they are used with `groupBy()` to perform summarization and computation over grouped data.

* Common Aggregation Functions in Spark

Function	Description	Example (Scala)	Example (SQL)
<code>count()</code>	Counts number of rows	<code>df.count()</code>	<code>SELECT count(*)</code>
<code>sum(col)</code>	Sum of values in column	<code>df.agg(sum("salary"))</code>	<code>SELECT sum(salary)</code>
<code>avg(col)</code>	Average of values	<code>df.agg(avg("salary"))</code>	<code>SELECT avg(salary)</code>
<code>min(col)</code>	Minimum value	<code>df.agg(min("salary"))</code>	<code>SELECT min(salary)</code>
<code>max(col)</code>	Maximum value	<code>df.agg(max("salary"))</code>	<code>SELECT max(salary)</code>
<code>first(col)</code>	First value	<code>df.agg(first("name"))</code>	<code>SELECT first(name)</code>
<code>last(col)</code>	Last value	<code>df.agg(last("name"))</code>	<code>SELECT last(name)</code>
<code>collect_list(col)</code>	Collects all values in a list	<code>df.agg(collect_list("name"))</code>	<code>SELECT collect_list(name)</code>
<code>collect_set(col)</code>	Collects distinct values in a set	<code>df.agg(collect_set("name"))</code>	<code>SELECT collect_set(name)</code>
<code>stddev(col)</code>	Standard deviation	<code>df.agg(stddev("salary"))</code>	<code>SELECT stddev(salary)</code>
<code>variance(col)</code>	Variance	<code>df.agg(variance("salary"))</code>	<code>SELECT variance(salary)</code>

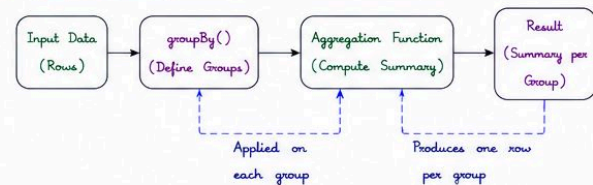
* Why Aggregation is Important?

- Provides meaningful insights from raw data.
- Essential for reporting, dashboards, and analytics.
- Reduces data size by summarizing large datasets.
- Enables efficient decision-making.

* Purpose / Role

- Facilitate summarization of large datasets.
- Compute statistics and metrics.
- Work on grouped data to provide insights (e.g., totals, averages, counts).
- Reduce data volume for efficient analysis.

* How it Works?



* Example - Grouped Aggregation

Input Data (employees)

emp_id	name	dept	salary
1	Amit	HR	50000
2	Bob	HR	60000
3	Cathy	IT	70000
4	David	IT	80000
5	Eva	IT	90000
6	Frank	Finance	55000

Query / Operation

Find average salary and total salary per department.

```
df.groupBy("dept")
  .agg(avg("salary").alias("avg_sal"),
       sum("salary").alias("total_sal"))
```

Result

dept	avg_sal	total_sal
HR	55000	110000
IT	80000	240000
Finance	55000	55000

* Spark SQL Example

```
SELECT dept,
       COUNT(*) AS emp_count,
       AVG(salary) AS avg_salary,
       SUM(salary) AS total_salary,
       MAX(salary) AS max_salary,
       MIN(salary) AS min_salary
FROM employees
GROUP BY dept;
```

* Key Points

- Aggregation functions are used with `groupBy()`.
- They return one row per group.
- Multiple aggregation functions can be applied simultaneously.
- Handles big data efficiently in a distributed manner.

Summary

Aggregation functions in Spark allow us to perform calculations over groups of data, producing summary results such as counts, sums, averages, minimums, maximums, etc. They are fundamental for data analysis and are widely used in business intelligence and machine learning workflows.

3. Inner and Outer Joins in Spark

What is a Join ?

A join is an operation that combines rows from two or more DataFrames based on a related column (key) between them.

In Spark, joins are transformations that return a new DataFrame containing columns from both DataFrames.

Inner Join vs Outer Join

Inner Join	Outer Join
• Returns only those existing rows from both DataFrames based on the join condition. • Rows without a match in the other DataFrame are excluded. • Result may have fewer rows.	• Returns all rows from one or both DataFrames. • If there is no match, NULL values are used for missing columns. • Result may have equal or more rows than the input DataFrames.

Sample DataFrames

DataFrame: employees (emp)

emp_id	name	dept_id	salary
1	Alice	101	50000
2	Bob	102	60000
3	Cathy	103	70000
4	David	104	80000

DataFrame: departments (dept)

dept_id	dept_name	location
101	HR	Delhi
102	Finance	Mumbai
104	Marketing	Bangalore
105	IT	Chennai

Note :

- Join condition used: `emp.dept_id = dept.dept_id`
- Column names may be different in real scenarios.

Key Points

- Inner Join returns only matching rows.
- Outer Joins return unmatched rows with NULLs.
- Use Left/Right when you want all data from one side.
- Use Full Outer when you need all data from both sides.
- Anti and Semi joins are useful for existence / non-existence checks.

Types of Joins in Spark

1. INNER JOIN

Returns only the matching rows from both DataFrames.

```
emp.join(dept, emp["dept_id"] == dept["dept_id"])
```

2. LEFT OUTER JOIN

Returns all rows from the left DataFrame (emp) and matching rows from the right (dept).

No match → NULLs for right side columns.

```
emp.join(dept, emp.dept_id == dept.dept_id, "left")
```

3. RIGHT OUTER JOIN

Returns all rows from the right DataFrame (dept) and matching rows from the left (emp).

No match → NULLs for left side columns.

```
emp.join(dept, emp.dept_id == dept.dept_id, "right")
```

4. FULL OUTER JOIN

Returns all rows from both DataFrames.

No match → NULLs for columns from the other side.

```
emp.join(dept, emp.dept_id == dept.dept_id, "outer")
```

5. LEFT ANTI JOIN

Returns rows from left DataFrame that do not have a match in the right DataFrame.

```
emp.join(dept, emp.dept_id == dept.dept_id, "left_anti")
```

6. LEFT SEMI JOIN

Returns rows from left DataFrame that have a match in the right DataFrame (only left columns).

```
emp.join(dept, emp.dept_id == dept.dept_id, "left_semi")
```

Result

emp_id	name	dept_id	salary	dept_name	location
1	Alice	101	50000	HR	Delhi
2	Bob	102	60000	Finance	Mumbai

Result

emp_id	name	dept_id	salary	dept_name	location
1	Alice	101	50000	HR	Delhi
2	Bob	102	60000	Finance	Mumbai
3	Cathy	103	70000	NULL	NULL
4	David	104	80000	Marketing	Bangalore

Result

emp_id	name	dept_id	salary	dept_name	location
1	Alice	101	50000	HR	Delhi
2	Bob	102	60000	Finance	Mumbai
4	David	104	80000	Marketing	Bangalore
NULL	NULL	105	NULL	IT	Chennai

Result

emp_id	name	dept_id	salary	dept_name	location
1	Alice	101	50000	HR	Delhi
2	Bob	102	60000	Finance	Mumbai
3	Cathy	103	70000	NULL	NULL
4	David	104	80000	Marketing	Bangalore
NULL	NULL	105	NULL	IT	Chennai

Result

emp_id	name	dept_id	salary
3	Cathy	103	70000

Result

emp_id	name	dept_id	salary
1	Alice	101	50000
2	Bob	102	60000
4	David	104	80000

4. Reading and Processing CSV, JSON, Text and Parquet Files using Spark's DataFrame API

Spark provides a unified DataFrame API to read data from different file formats, perform transformations, and write the results. The DataFrame API automatically handles schema inference, optimization and distributed processing.



1. CSV Files

CSV (Comma Separated Values) is a simple tabular format where each line represents a row and values are separated by commas.

Example (employees.csv)

```
id,name,dept,salary
1,Alice,HR,50000
2,Bob,Finance,60000
3,Cathy,IT,70000
```

Read CSV

```
df_csv = spark.read.option("header", True) \
    .option("inferSchema", True) \
    .option("delimiter", ",") \
    .csv("/data/employees.csv")
```

Process

```
df_csv.filter(df_csv["salary"] > 55000).show()
```

- Advantages
- Human readable
 - Widely supported
 - Simple and portable
 - Good for small to medium datasets

2. JSON Files

JSON (JavaScript Object Notation) is a semi-structured format with key-value pairs and nested structures.

Example (employees.json)

```
[
  {"id": 1, "name": "Alice",
   "dept": "HR", "salary": 50000},
  {"id": 2, "name": "Bob",
   "dept": "Finance", "salary": 60000},
  {"id": 3, "name": "Cathy",
   "dept": "IT", "salary": 70000}
]
```

Read JSON

```
df_json = spark.read.option("inferSchema", True) \
    .json("/data/employees.json")
```

Process

```
df_json.select("name", "dept").where("salary > 60000").show()
```

- Advantages
- Supports nested and hierarchical data
 - Self-describing (schema is embedded)
 - Ideal for semi-structured data and NoSQL sources
 - Easy integration with web applications
- Key Points
- Use DataFrameReader (spark.read) with appropriate format-specific options.
 - Perform transformations using DataFrame operations.
 - Choose file format based on data size, structure, and performance needs.

3. Text Files

Text files contain plain text data, one record per line (no structure).

Example (employees.txt)

```
1|Alice|HR|50000
2|Bob|Finance|60000
3|Cathy|IT|70000
```

Read Text

```
df_text = spark.read.option("delimiter", "|") \
    .option("inferSchema", True) \
    .csv("/data/employees.txt")
```

Process

```
df_text.groupBy("dept").count().show()
```

- Advantages
- Very simple format
 - Lightweight
 - Useful for logs and unstructured data
 - Easy to generate and append

4. Parquet Files

Parquet is a columnar, binary file format optimized for big data processing. It stores schema with data and supports compression and predicate pushdown.

Example (employees.parquet)

(Binary columnar file - not human readable)

Read Parquet

```
df_parquet = spark.read.parquet("/data/employees.parquet")
```

Process

```
df_parquet.filter("salary > 60000").groupBy("dept").count().show()
```

- Advantages
- Columnar storage for fast analytics
 - High compression → less storage
 - Supports schema evolution
 - Efficient for big datasets
 - Works well with Spark SQL and Hive

Summary Comparison

Format	Structure	Human Readable	Schema	Best Use Case	Advantages
CSV	Tabular (rows & cols)	Yes	Optional	Small-Medium, Data Exchange	Simple, Portable, Widely supported
JSON	Semi-structured (key-value)	Yes	Embedded / Inferable	Nested data, APIs, NoSQL	Flexible, Self-describing, Handles nested data
Text	Unstructured / Delimited	Yes	None	Logs, raw data, Streaming	Simple, Lightweight, Easy to append
Parquet	Structured (Columnar)	No	Embedded	Large scale analytics, Data warehouse	Fast, Compressed, Optimized for Spark

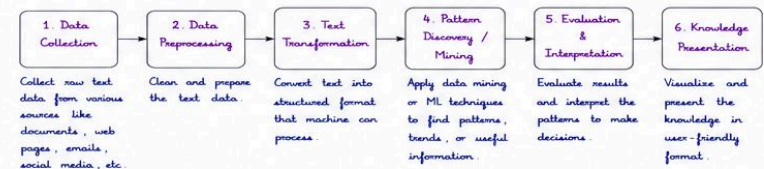
1. Batch Processing vs Stream Processing and Text Mining Process

A. Batch Processing vs Stream Processing

Aspect	Batch Processing	Stream Processing
Definition	Processes large volumes of data stored in batches at rest.	Processes data continuously as it is generated.
Data Handling	Data is collected, stored and then processed.	Data is processed in real-time as a continuous flow.
Latency	High latency (results available after entire batch is processed).	Low latency (results available almost instantly).
Processing Model	Scheduled jobs (e.g., hourly, daily).	Continuous and event-driven processing.
Data Size	Suitable for large historical data.	Suitable for high velocity and high volume data.
Fault Tolerance	Relies on re-running jobs on failure.	Requires mechanisms like checkpoints, offsets, exactly-once processing.
Examples	Payroll processing, monthly sales report, data warehousing.	Fraud detection, stock market analytics, IoT sensor monitoring, real-time dashboards.
Tools (Spark)	Spark Core, Spark SQL (batch).	Spark Streaming, Structured Streaming.

C. Text Mining Process (Steps)

Text Mining is the process of extracting useful and meaningful information from unstructured text data.



Details of Each Step :

- Data Collection** : Gather text data from sources (files, databases, APIs, web scraping).
- Data Preprocessing** : Remove noise and make text clean.
 - Lowercasing
 - Removing punctuation, numbers, special characters
 - Removing stop words (e.g., the, is, at)
 - Stemming / Lemmatization (e.g., "running" → "run")
 - Handling missing or empty data.
- Text Transformation** : Convert text into numerical or structured representation.
 - Tokenization (split into words or phrases)
 - Vectorization (Bag of Words, TF-IDF, Word Embeddings)
 - Creating Document-Term Matrix.

B. When is Stream Processing More Suitable?

- When real-time insights and immediate action are required.
e.g., Fraud detection in banking, intrusion detection in networks.
- When data is generated continuously and cannot be stored.
e.g., IoT devices, sensors, clickstreams.
- When low latency is critical for decision making.
e.g., Live dashboards, dynamic pricing, advertising.
- When handling high velocity and high volume data.
e.g., Social media feeds, stock market data.
- When systems need to react to events as they occur.
e.g., Alert systems, personalized recommendations.

- Pattern Discovery / Mining** : Apply techniques to extract useful patterns.
 - Classification (e.g., spam vs not spam)
 - Clustering (e.g., grouping similar documents)
 - Topic Modeling (e.g., LDA)
 - Sentiment Analysis (e.g., positive, negative, neutral)
 - Association Rule Mining, etc.
- Evaluation & Interpretation** : Check accuracy, precision, recall, coherence (for topics), etc., and interpret the results in business context.
- Knowledge Presentation** : Present the findings through dashboards, reports, charts, word clouds, alerts, or recommendations.

Example Application of Text Mining

- Customer feedback analysis
- Email spam filtering
- News classification
- Social media sentiment analysis
- Document summarization



2. Overview of Structured Streaming in Apache Spark

Structured Streaming is Spark's scalable and fault-tolerant stream processing engine built on the Spark SQL engine. It allows you to process real-time data as a continuously arriving table (stream) using the same high-level APIs you use for batch data.

Key Capabilities

- Unified API: Use the DataFrame/Dataset API for both batch and streaming.
- High-level abstraction: Treat streaming data as a table.
- Fault tolerance: Automatic recovery using write-ahead logs and checkpointing.
- Exactly-once processing: End-to-end exactly-once guarantees (with supported sinks).
- Scalability: Horizontal scalability with micro-batch execution.
- Rich integrations: Sources (Kafka, files, sockets, etc.) and sinks (Kafka, files, DBs, etc.).

Unified Abstraction for Batch and Stream Processing

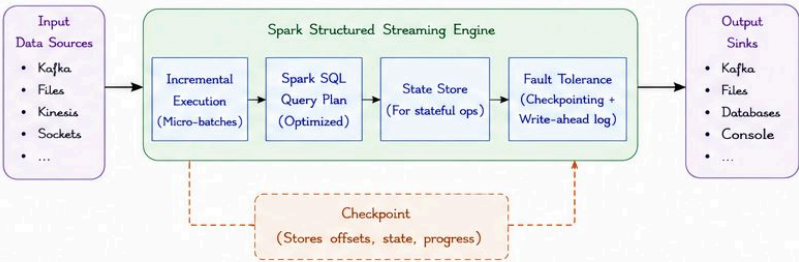
Structured Streaming represents both batch and streaming data as a DataFrame. The same operations (select, filter, join, groupBy, aggregation, etc.) and the same APIs are used for both.

Aspect	Batch Processing	Stream Processing (Structured Streaming)
Input	Finite dataset (e.g., files, tables)	Unbounded dataset (e.g., Kafka topic)
Data Representation	DataFrame	DataFrame (streaming)
Execution	Single execution over entire data	Incremental execution in micro-batches
Result	Static result	Continuously updated result
API	DataFrame/Dataset API	Same DataFrame/Dataset API
Fault Tolerance	Not required	Checkpointing + Write-ahead logs
Examples	ETL jobs, reports	Real-time analytics, monitoring, alerts

Key Components

- Sources: Continuously provide new data. (Kafka, files, sockets, rate source, etc.)
- Streaming DataFrame: Unbounded DataFrame representing the input as a stream.
- Transformations: Same operations as batch (stateless or stateful).
- Sinks: Output the results continuously.
- Checkpoint: Directory to store offsets, state, and metadata for fault tolerance.
- Trigger: Determines how often the micro-batch runs.
 - ProcessingTime: Every N seconds (default)
 - Once: Process all available data and stop
 - Continuous: As data arrives (low latency, specialized sources/sinks)

How Structured Streaming Works (Micro-batch Execution)



Example: Same Query on Batch and Streaming Data

Compute running count of clicks by country

Batch DataFrame

```
df = spark.read.parquet("/data/clicks/")
result = (df.groupBy("country")
          .count())
result.show()
```

Streaming DataFrame

```
df = (spark.readStream
      .format("kafka")
      .option("subscribe", "clicks")
      .load()
      .selectExpr("CAST(value AS STRING) as json")
      .select(from_json(col("json"), schema.*)))

result = (df.groupBy("country")
          .count())

query = (result.writeStream
          .outputMode("complete")
          .format("console")
          .start())
```

The same transformation (groupBy + count) is used. Only the data source and the way results are written differ.

Advantages

Advantage	Description
Unified Programming Model	One API for both batch and stream; easy to learn and maintain.
Code Reuse	Same code works on historical data (batch) and real-time data (stream).
Scalability	Built on Spark's distributed engine; scales to large data and high throughput.
Fault Tolerance	Checkpointing and WAL provide exactly-once guarantees.
Expressive & SQL-friendly	Leverages DataFrame/Dataset and Spark SQL for complex analytics.
Low Latency	Micro-batch processing (sub-second latencies supported).

3. Kafka as a Popular Message Broker and its Role in Data Stream Processing

A. Overview of Apache Kafka

Apache Kafka is a distributed, high-throughput, fault-tolerant message broker (event streaming platform) originally developed by LinkedIn and now an Apache Software Foundation project. Kafka is designed to handle real-time data feeds at scale with high durability and low latency.

Key Features

- Publish-subscribe messaging model.
- High throughput and low latency.
- Scalable and distributed (horizontal scale).
- Durable messages with configurable retention.
- Fault tolerant: replicated and partitioned data.
- Ordered messages within a partition.
- Supports multiple consumers in a consumer group.
- Persistent storage on disk (not in memory).
- Exactly-once semantics (with idempotent producers and transactions).

D. How Kafka Works (Flow)



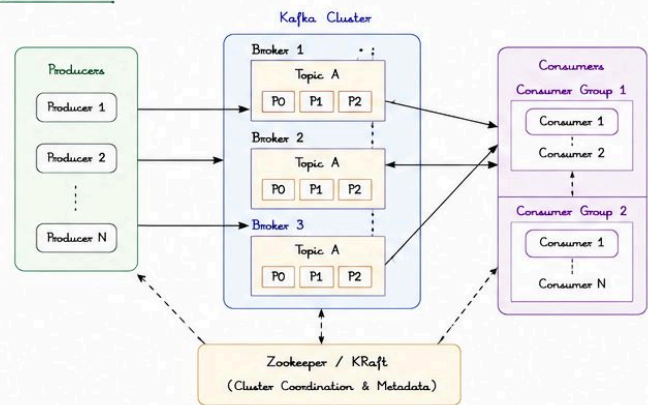
E. Kafka Guarantees

- Durability: Messages are persisted to disk and replicated.
- Availability: Data is available even if some brokers fail.
- Ordering: Guaranteed within a partition.
 - At most once
 - At least once (default)
 - Exactly once (with idempotent producer + transactions)

B. Core Concepts

Concept	Description
Producer	Application that publishes (writes) messages to a topic.
Topic	A named stream/category of records. Topics are divided into partitions.
Partition	An ordered, immutable sequence of messages. Provides parallelism and scalability.
Broker	A Kafka server that stores data and serves clients.
Cluster	A set of one or more brokers working together.
Consumer	Application that subscribes to topics and reads messages.
Consumer Group	A set of consumers that cooperatively consume a topic.
Offset	A unique ID for each message in a partition; consumers track offsets (committed position).
Zookeeper / KRaft	(Earlier) ZooKeeper manages metadata and coordination. (Newer) KRaft mode removes ZooKeeper.

C. Kafka Architecture

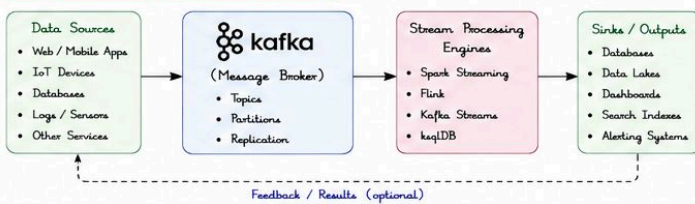


G. Role in Data Stream Processing

Kafka acts as the backbone of modern data stream architectures by providing a reliable, scalable, and durable pipeline for streaming data.

- Ingests data from multiple sources (apps, devices, logs, databases).
- Decouples producers and consumers (asynchronous processing).
- Buffers data and absorbs traffic spikes.
- Integrates with stream processing engines like:
 - Apache Spark Structured Streaming
 - Apache Flink
 - Kafka Streams
 - ksqldb
- Enables real-time processing, analytics, and machine learning.

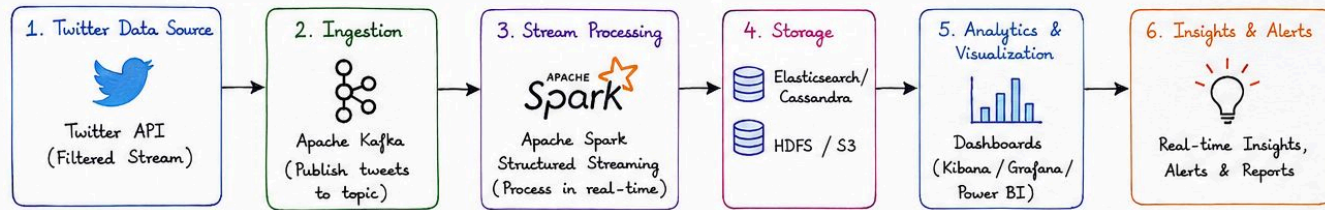
H. Typical Streaming Architecture with Kafka



F. Common Use Cases

- Real-time analytics and monitoring
- Log aggregation
- Event-driven microservices
- Data pipelines and ETL
- IoT data ingestion
- Stream processing applications

Ingesting Live Twitter Data, Processing it in Real-time and Generating Insights



End-to-End Steps :

1. Connect to Twitter API

- Create a Twitter Developer account and app.
- Use Twitter API v2 (Filtered Stream endpoint).
- Define rules/keywords/hashtags to track (e.g., #AI, #ClimateChange).

2. Ingest Data with Kafka

- A producer application (Tweepy, Kafka Connector, or custom Python/Java app) reads tweets.
- Tweets are published to a Kafka topic (e.g., tweets_raw).
- Kafka provides durability, buffering and decouples producer and consumers.

3. Process in Real-time with Spark Structured Streaming

- Spark reads data from Kafka topic as a streaming source.
- Perform real-time transformations and enrichment.
- Typical operations: parse JSON, clean text, detect language, sentiment analysis, extract entities, filter spam, derive metrics (counts, trends) etc.

4. Store Data for Fast Access & History

- Save processed data to a fast store for real-time queries (Elasticsearch/Cassandra).
- Save raw and processed data to data lake for historical analysis (HDFS/S3/Delta Lake).

5. Visualize & Monitor

- Build dashboards using Kibana, Grafana or Power BI.
- Visualize metrics such as tweet volume, top topics, sentiment over time, locations, popular users, etc.
- Monitor pipeline health and data quality.

6. Generate Insights & Alerts

- Derive insights from dashboards and analytics.
- Set up alerts for spikes in volume, negative sentiment, or specific keywords.
- Deliver reports to stakeholders via email, Slack, or web dashboards.

Examples of Real-time Insights :



Trending Topics

Identify top hashtags/keywords rising in real time.



Sentiment Analysis

Understand public sentiment (positive, negative, neutral) over time.



Geospatial Insights

Map tweet volume or sentiment by location.



Influencer Detection

Find most active or most influential users.



Anomaly & Alerting

Detect unusual spikes, crises, or breaking news.

Typical Tech Stack :

Data Source	Twitter API v2 (Filtered Stream)
Ingestion	Apache Kafka
Stream Processing	Apache Spark Structured Streaming
Storage	Elasticsearch / Cassandra, HDFS / S3
Visualization	Kibana / Grafana / Power BI
Alerts	Alertmanager / Email / Slack
Orchestration	Apache ZooKeeper, Docker, Kubernetes

Flow Summary :

